SIH 2026 — MoSPI / PAIMANA

AI-Powered Predictive Analytics & Early Warning System for Infrastructure Project Monitoring

Detailed Solution Approach & Literature Review Reading Guide

Based on synthesis of 100 peer-reviewed papers on AI/ML for cost & schedule overrun prediction, risk scoring, and early-warning systems in infrastructure/construction project management.

Contents

1. Executive Summary

This document translates a synthesis of 100 research papers (2020–2026) on AI/ML for construction and infrastructure project performance into a concrete, phased build plan for the SIH 2026 problem statement on PAIMANA. The literature consistently shows that ensemble tree-based models (XGBoost, Random Forest, LightGBM, CatBoost) and their hybrids/stacks are the strongest performers for cost and schedule overrun prediction, that Bayesian/Markov-based models are best suited to time-varying early warning, that Large Language Models are emerging as a strong tool for extracting risk signals from unstructured text rather than for numeric prediction, and that simpler methods such as Reference Class Forecasting remain a credible, low-data-requirement benchmark that AI must be shown to beat — directly answering MoSPI's explicit question of whether AI/ML provides real gains over conventional statistical methods.

The recommended solution is a modular pipeline: (1) a structured-data ensemble model for cost/time overrun prediction and risk scoring, (2) a Bayesian/HMM-based module for continuous, monthly-updated early warning, (3) an LLM-based module for mining free-text/unstructured project remarks for risk signals not captured in the Common Upload Form (CUF), and (4) an explainable dashboard (SHAP-based) that ties every alert back to its underlying drivers, all built on open-source tools.

2. Detailed Recommended Approach

Phase 1 — Data Acquisition & Feature Engineering

Objective: Build a clean, well-structured, historically deep dataset that supports both retrospective model training and monthly-updated inference.

1.1 Data Sources

- Primary: PAIMANA monthly Project Monitoring Reports (structured CUF fields) — https://paimana-proj.mospi.gov.in/ReportPage. Start with the April 2026 report referenced in the problem statement, then backfill as many prior months as accessible.
- Historical: Legacy OCMS records (up to ~20 years) if accessible through MoSPI/IPMD, to give the model a longer time horizon and more overrun/delay events to learn from.
- External enrichment (to test dimension 'c' of the problem statement — CUF vs. additional variables): wholesale price index / construction input inflation (for material cost escalation), rainfall/weather data by project region (IMD), sector-level GDP or capex trend, exchange rate data for import-heavy sectors (e.g., mining/steel equipment).

1.2 Feature Engineering (grounded in literature findings)

The following variables recur as the strongest predictors across the reviewed papers and should be prioritized:

- Project size / original sanctioned cost (strong predictor in nearly every cost-overrun study reviewed)
- Project duration (planned) and elapsed duration to date
- Sector / Ministry / implementing agency (captures systemic differences — e.g., South Asian benchmark average overrun was only ~3.3%, showing large regional/sectoral variance)
- Contract / procurement type (design-build vs. design-bid-build changes overrun magnitude)

- Revised-cost-to-original-cost ratio trend (month-on-month) — a leading indicator, not just an outcome
- Milestone slippage history (number and size of past delays for the same project)
- Physical progress vs. financial progress gap (a classic EVM-style signal used across many reviewed models)
- Scope-change / variation-order frequency (flagged as the largest single cause of deviation — up to 76% of triggers in one study)
- Contractor track record across other PAIMANA-tracked projects, if derivable

Cross-study validation: a focused review of 20 head-to-head ML-vs-EVM studies confirms the same top predictors recur independently — variation orders/scope changes (41% feature importance in one 836-project study), initial contract amount, project duration, and physical scale (e.g., number of storeys) are repeatedly the strongest signals. This convergence across unrelated datasets and countries is strong justification for prioritizing these fields first in your PAIMANA feature set.

1.3 CUF Sufficiency Test

Explicitly build two parallel feature sets — (A) CUF-only fields, and (B) CUF + externally enriched variables — and compare model performance on each. This produces the direct, defensible answer to MoSPI's requirement (c): quantifying how much predictive lift comes from data not currently captured in the CUF, and making a concrete recommendation on which new fields (if any) are worth adding to the form.

Phase 2 — Baseline & Comparative Modeling

Objective: Establish honest, defensible baselines before introducing complex ML, directly addressing MoSPI's requirement (b) — whether AI/ML provides significant gains over conventional statistical methods.

- Baseline 1 — Naive/trend extrapolation: simple linear trend on revised cost / schedule slippage.
- Baseline 2 — Multiple linear regression on CUF fields.
- Baseline 3 — Reference Class Forecasting (RCF): group historical PAIMANA projects into reference classes (by sector, size band, agency) and use the empirical overrun distribution of similar past projects as the forecast. This method cut UK infrastructure overruns from 38% to 5% after adoption and is the single most policy-credible non-ML benchmark in the literature — MoSPI evaluators are very likely to respect a submission that treats this seriously rather than skipping straight to deep learning.
- Document accuracy (MAPE, RMSE, R^2) and business-relevant metrics (e.g., % of high-risk projects correctly flagged 3+ months before overrun) for every baseline, so the eventual ML gain (or lack of it) can be reported honestly.

Phase 3 — Advanced Predictive Models (Cost & Schedule)

3.1 Primary Model Family: Gradient-Boosted Ensembles

Across the reviewed literature, XGBoost, LightGBM, CatBoost, and Random Forest are the most consistently high-performing models for both cost overrun and schedule delay prediction, frequently reaching 85%+ accuracy or R^2 above 0.9 on project-level datasets of comparable scale to PAIMANA's ~2,000 tracked projects.

- Train separate models for: (a) cost overrun magnitude (regression), (b) cost overrun risk class — e.g., low/medium/high (classification), (c) schedule delay in months (regression), (d) delay risk class (classification).

- Use time-aware train/validation/test splits (train on projects/months up to a cutoff, validate on the next period) rather than random splits, since PAIMANA data updates monthly and the real use case is forward prediction.

3.2 Stacked / Hybrid Ensemble (recommended stretch goal)

The strongest results in the literature come from stacking multiple base learners (e.g., XGBoost + LightGBM + CatBoost + Random Forest) with a simple meta-learner (Ridge Regression) on top — one reviewed 2026 study achieved $R^2 = 0.971$ this way, clearly outperforming any single base model. This is a realistic, implementable stretch goal using open-source scikit-learn/XGBoost/LightGBM/CatBoost stacks.

3.3 When to Consider Deep Learning

Deep learning (ANNs, LSTMs, or spatio-temporal graph networks) tends to outperform tree ensembles only when there is a large volume of sequential/time-series data per project (e.g., monthly progress curves over many years) and complex nonlinear interactions. Given PAIMANA's monthly cadence and moderate project count, deep learning should be treated as a secondary experiment, not the primary model — tree ensembles are more data-efficient and far more interpretable, which matters for a government decision-support tool.

Phase 4 — Risk Scoring & Time-Varying Early Warning

Objective: Move beyond a single static prediction to a continuously updating early-warning signal, since PAIMANA refreshes monthly and risk should evolve as new CUF data arrives.

- Use a Bayesian Network or Hidden Markov Model layer on top of the ensemble outputs to model risk state transitions (e.g., Low → Watch → High-Risk → Critical) month over month, rather than a one-shot classification. Multiple reviewed papers use exactly this approach for early-alert cost overrun systems.
- Combine cost-risk probability, schedule-risk probability, and causal-factor flags (scope-change frequency, milestone slippage, financial-vs-physical progress gap) into a single composite Project Risk Score (e.g., 0–100), with the sub-scores retained and shown separately for transparency.
- Define clear, documented alert thresholds and escalation logic (e.g., score crossing a threshold for two consecutive months triggers an alert to the concerned Ministry/monitoring agency) — this operationalizes the 'Early Warning Alert System' outcome MoSPI explicitly lists.

Phase 5 — LLM-Enabled Project Intelligence Layer

Objective: Extract risk signals from unstructured/free-text data that the structured CUF fields miss — this is the most novel, least-crowded part of the literature and directly matches MoSPI's 'LLM-enabled Project Intelligence Assistant' outcome.

- Sources: free-text remarks/comments fields in CUF submissions, correspondence or minutes-of-meeting text if available, news/media mentions of specific projects (open web sources) for early signals of contractor disputes, land acquisition issues, or litigation.
- Use an open-source LLM (e.g., Llama-family or Mistral, run locally or via an open-source inference server) with zero-shot or few-shot prompting to classify text into standard risk categories (financial, contractual, land/environmental, resource, safety). The literature shows GPT-4-class few-shot performance ($F1 \approx 0.81$) is competitive with a trained BERT+SVM classifier ($F1 \approx 0.86$) without requiring labeled training data — making this approach very feasible on a hackathon timeline with open-source models.

- Build a natural-language query interface ('Project Intelligence Assistant') so administrators can ask questions like 'which projects in the Transport sector have the highest combined cost and schedule risk this month, and why?' and get an answer grounded in the structured risk scores plus the LLM-extracted qualitative flags — combining Phases 3, 4, and 5 into one usable tool.
- Guardrail: keep the LLM layer advisory and explanation-generating, not the sole basis for numeric risk scores — numeric predictions should come from the auditable ensemble models in Phase 3, with the LLM adding interpretive/contextual value.

Phase 6 — Dashboard, Explainability & Deployment

- Use SHAP (SHapley Additive exPlanations) on every ensemble prediction so each flagged project shows its top 3–5 contributing risk drivers in plain language, not just a score — this is now treated as a baseline requirement in the most recent (2025–2026) papers in the corpus, not an optional extra.
- Dashboard views: (a) portfolio-level heat map of risk across ministries/sectors, (b) project drill-down with risk trend over time and driver breakdown, (c) early-warning alert feed, (d) benchmarking view comparing a project against its 'reference class' peers.
- Build with open-source stack: Python (pandas, scikit-learn, XGBoost/LightGBM, SHAP) for modeling; a lightweight open-source dashboard framework (e.g., Streamlit, Dash, or a React + open-source charting library) for the front end; PostgreSQL or similar for storage.
- Documentation & deployment framework: include a data dictionary mapping every model feature back to its CUF field (or flag it as a new field to be added), a model card describing training data, performance, and limitations, and a simple retraining/refresh procedure tied to PAIMANA's monthly update cycle — this directly satisfies the 'Documentation and deployment framework' outcome MoSPI lists.

Phase 7 — Evaluation Framework (ML vs. Conventional Methods)

This phase directly answers MoSPI's explicit technical question (b) and should be presented as a first-class result, not an afterthought.

- Report accuracy/error metrics for every method side by side: naive baseline, linear regression, Reference Class Forecasting, and each ML model — using the same train/test split and the same evaluation window.
- Report not just point-accuracy but early-warning lead time: how many months in advance each method correctly flags an eventual overrun/delay, since this is the metric that actually matters for intervention.
- Be transparent where simpler methods are competitive — several reviewed studies found actuarial/RCF-style methods matched or beat ML in certain project categories (e.g., smaller or more homogeneous portfolios). Reporting this honestly, rather than only showcasing ML wins, strengthens the credibility of the submission and directly matches what MoSPI asked for.

2.7.1 Quantified Evidence: ML vs. EVM/Regression (Head-to-Head Studies)

A focused review of 20 additional head-to-head comparison studies gives concrete, citable numbers for this exact question — use this table directly in your evaluation report and pitch deck rather than a general claim that 'ML is better':

Study	Best ML Model	Result	Comparison to Regression/EVM

ForouzeshNejad et al. (2024)	XGBoost-SA (hybrid)	92% accuracy	~50% lower cost error, ~80% lower time error than EVM/ESM
Yalcin, Bayram & Çıtakoğlu (2024)	Classical ANN	Best OI/NSE scores	Outperformed 19 EVM-based cost methods using current ML approaches
Elmousalami (2019)	XGBoost	MAPE 9.09%, adj. R^2=0.929	Best of 20 AI/statistical techniques compared, incl. multiple regression
Rajkumar et al. (2026)	Random Forest	R^2=0.80–0.87 by sector	Ridge & Linear Regression trailed all ML models tested
Hamdan & Thneibat (2025)	CatBoost	R^2=0.883 (836 projects)	Large-scale validation; regression not competitive at this data volume
Moon & Williams (2020)	Ensemble classifier	Only 61.41% accuracy	Caveat: ML advantage shrinks sharply at the early bidding stage with sparse data

Key nuance for your pitch: the ML-vs-conventional gap is real and large where data is sufficient (50–80% error reduction in the strongest studies), but narrows sharply at the early/sparse-data stage (bidding stage accuracy of just 61.4% in Moon & Williams). This matches PAIMANA's reality — newly sanctioned projects will have less history than mature ones — and gives you a defensible, honest answer to 'does ML always win' rather than an overclaim.

3. Risks, Limitations & How to Address Them

- Data volume/history: if only a few months of PAIMANA data are accessible during the hackathon, supplement with OCMS historical records and clearly document the limitation; favor data-efficient tree ensembles over deep learning.
- Class imbalance: severe cost/time overruns are relatively rare events; use techniques such as class weighting, SMOTE, or focal-loss-style objectives, and evaluate with precision/recall/F1 on the minority (high-risk) class, not just overall accuracy.
- Interpretability requirement: government stakeholders need to trust and act on outputs — prioritize SHAP-explained tree ensembles and Bayesian Networks over black-box deep nets unless a clear accuracy gain justifies the interpretability trade-off.
- Data quality/reporting inconsistency across ministries — build a validation/cleaning layer as an explicit pipeline stage, and quantify data-quality issues as a finding in itself.
- Avoid overclaiming LLM capability: use the LLM for qualitative risk-flagging and explanation generation, not as an unverified source of numeric predictions.

- Early-stage/sparse-data underperformance: for newly sanctioned projects with little history, ML accuracy can drop sharply (one bidding-stage study saw accuracy fall to ~61%) — for such projects, blend ML output with the Reference Class Forecasting baseline rather than relying on ML alone until more monthly data points accumulate.

4. Complete Technology Stack (Mapped to Each Phase)

Every tool below is free and open-source unless explicitly marked otherwise, directly satisfying MoSPI's requirement to use only open-source tools/software. Where a reviewed paper used a closed/proprietary tool (e.g., GPT-4), an open-source equivalent is given as the default recommendation, with the proprietary option noted only as an optional stretch/comparison.

4.1 Technology Map by Phase

Phase / Task	Technology	License / OSS Status
Data ingestion & scraping	Python 3.x, requests, BeautifulSoup4, Scrapy (if PAIMANA needs scraping)	Open source (PSF / MIT / BSD)
Data pipeline / scheduling	Apache Airflow or a simple cron + Python script (for monthly PAIMANA refresh)	Open source (Apache 2.0)
Data storage	PostgreSQL (primary) or SQLite (lightweight/prototype)	Open source (PostgreSQL License / Public Domain)
Data manipulation	pandas, NumPy	Open source (BSD)
Data cleaning / validation	pandas, Great Expectations (optional, for CUF data-quality checks)	Open source (Apache 2.0)
Feature engineering	scikit-learn pipelines, pandas, category_encoders	Open source (BSD)
Baseline statistical models	statsmodels (linear regression), scikit-learn	Open source (BSD)
Reference Class Forecasting logic	Custom Python (pandas groupby + empirical distributions) — no special library needed	Open source (your own code)
Core ML — cost/schedule prediction	XGBoost, LightGBM, CatBoost, scikit-learn (Random Forest)	Open source (Apache 2.0 / MIT)
Hybrid/stacked ensembles	scikit-learn StackingRegressor/Classifier on top of the above	Open source (BSD)
Deep learning (optional, Phase 3.3)	PyTorch or TensorFlow/Keras (for ANN/LSTM experiments)	Open source (BSD / Apache 2.0)
Bayesian Network / HMM (early warning)	pgmpy (Bayesian Networks), hmmlearn (Hidden Markov Models)	Open source (MIT / BSD)
Model explainability	SHAP, ELI5	Open source (MIT)

Experiment tracking / versioning	MLflow	Open source (Apache 2.0)
LLM — text risk classification & Q&A assistant	Llama 3.1/3.2 or Mistral 7B, run via Ollama or Hugging Face Transformers	Open source (Llama Community License / Apache 2.0)
LLM orchestration / RAG	LangChain or LlamaIndex, with FAISS or ChromaDB as vector store	Open source (MIT / Apache 2.0)
Text embeddings	sentence-transformers (e.g., all-MiniLM-L6-v2)	Open source (Apache 2.0)
Backend / model serving API	FastAPI (Python)	Open source (MIT)
Dashboard / front-end	Streamlit or Dash (fastest to build) — or React + Recharts/Plotly.js for a more polished UI	Open source (Apache 2.0 / MIT)
Visualization (charts, heat maps)	Plotly, Matplotlib, Seaborn	Open source (MIT / BSD)
Containerization / deployment	Docker, Docker Compose	Open source (Apache 2.0)
Version control & collaboration	Git, GitHub (public/free tier)	Open source (Git) / free-tier platform (GitHub)
Documentation	Markdown, MkDocs or Sphinx for the model card / deployment docs	Open source (BSD / MIT)

4.2 Where Each Technology Is Used (Walkthrough)

- **Data layer:** Python scripts pull/parse PAIMANA's monthly reports and any historical OCMS exports; pandas cleans and reshapes them; PostgreSQL stores the structured CUF-field history so models can query point-in-time snapshots (important for the time-aware train/test splits in Phase 3).
- **Baseline layer:** statsmodels/scikit-learn build the linear regression baseline; a small custom Python module implements Reference Class Forecasting by grouping projects into peer classes and computing empirical overrun distributions — this is your Phase 2 conventional-method benchmark.
- **Prediction layer:** XGBoost/LightGBM/CatBoost/Random Forest (via scikit-learn's unified API) train the cost and schedule overrun models; a StackingRegressor combines them for the Phase 3.2 hybrid ensemble; SHAP explains every prediction.
- **Early-warning layer:** pgmpy builds the Bayesian Network that models interdependent risk factors; hmmlearn models risk-state transitions (Low→Watch→High-Risk→Critical) month over month as new CUF data lands — this is Phase 4.
- **LLM layer:** Ollama serves an open-source LLM (Llama 3.1 8B or Mistral 7B) locally, with no API cost and no data leaving your environment — important for government data. LangChain orchestrates prompts for risk classification of free-text CUF remarks and powers the natural-language 'Project Intelligence Assistant' Q&A

interface, retrieving relevant structured risk scores via a simple RAG (retrieval-augmented generation) setup with FAISS.

- **Serving & dashboard layer:** FastAPI exposes model predictions and risk scores as REST endpoints; Streamlit (fastest for a hackathon) or a React+Plotly front-end consumes these endpoints to render the portfolio heat map, project drill-downs, and the LLM assistant chat interface.
- **Packaging & docs layer:** Docker Compose bundles the database, API, and dashboard into one runnable stack for judges to try; MkDocs/Markdown produces the documentation and model card MoSPI explicitly lists as a desired outcome.

4.3 Open-Source Compliance Note

Every core component above (data processing, all ML models, explainability, Bayesian/HMM modeling, the LLM itself, the backend, and the dashboard) is fully open-source, with no paid API required to run the system end-to-end. The only place the literature you reviewed used a closed model was GPT-4 for LLM risk classification (Erfani & Khanjar, 2025) — for your build, Llama 3.1/Mistral run via Ollama is the default, since the same paper shows open few-shot LLM performance (F1≈0.81) is already competitive with a trained classical classifier, and using an open model keeps you fully compliant with the problem statement's explicit preference for open-source tools. If you want an accuracy comparison point for your evaluation section, you may optionally call a proprietary API (e.g., GPT-4) once as a benchmark row in a table — but the deployed system itself should run entirely on the open-source stack above.

5. The Winning Roadmap

Honest caveat first: no roadmap can guarantee a win — SIH judging involves multiple teams, subjective panels, and factors outside your control. What this section gives you instead is the highest-probability path to a strong result: a plan built directly from what SIH judges consistently reward, mapped against what the literature review shows is technically credible and achievable in hackathon time. Execute this fully and you will have a genuinely competitive, defensible submission.

5.1 How SIH Submissions Are Actually Judged

Across SIH cycles, panels consistently weight these dimensions — build your plan backward from them, not forward from the tech stack:

- Innovation / uniqueness of approach — not 'used XGBoost' but a specific, defensible idea (e.g., the CUF-sufficiency test, or the LLM Intelligence Assistant) that other teams tackling the same PS are unlikely to have.
- Technical feasibility & depth — can you show it actually works on real or realistic data, not just slides.
- Relevance to the exact problem statement — explicit coverage of every technical dimension MoSPI listed (a, b, c) and every suggested outcome, even briefly.
- Usability & presentation — a clean, working demo/dashboard beats a more accurate model with no interface.
- Scalability & deployment realism — evidence you understand how this plugs into an actual government system (PAIMANA), not just a Kaggle-style notebook.
- Impact & clarity of communication — can a non-technical judge understand the value in under 2 minutes.

5.2 Your Differentiation Strategy

Many teams will pick this problem statement and most will converge on 'we used Random Forest/XGBoost to predict cost overruns.' To stand out, lead with what the literature shows is under-explored and directly asked for by MoSPI:

- Own the CUF-sufficiency analysis (Phase 1.3) as your headline finding — a clear, quantified answer to 'does AI need more data than CUF currently collects, and if so, exactly what.' This is a direct, explicit ask in the problem statement that most teams will skip because it requires rigor, not just a model.
- Own the honest ML-vs-conventional-methods comparison (Phase 7) — most teams will only show their best model's accuracy. Showing Reference Class Forecasting and regression baselines side-by-side, and being candid about where ML does/doesn't help, signals maturity and directly answers requirement (b).
- Own the LLM Project Intelligence Assistant as your demo centerpiece — it's visually impressive in a live demo (natural-language Q&A over project risk data), technically current, and explicitly named as a desired outcome, but requires more NLP know-how than most teams will attempt.
- Own the explainability layer (SHAP-driven 'why is this project flagged') — judges consistently respond well to seeing a model justify itself in plain language, since it signals real-world deployability for a government audience.

5.3 Execution Timeline (Compress/Expand to Your Actual Hackathon Calendar)

Stage	Focus	Deliverable at End of Stage
Week 1	Data acquisition, cleaning, CUF-field mapping, exploratory analysis; finalize baselines (linear regression, RCF)	Clean dataset + baseline accuracy numbers
Week 2	Feature engineering (incl. enrichment variables); train core ensemble models (XGBoost/LightGBM/RF) for cost & schedule	Working cost/schedule prediction models with metrics
Week 3	Build risk scoring + Bayesian/HMM early-warning layer; run CUF-sufficiency and ML-vs-conventional comparisons	Composite risk score + evaluation tables (Phases 4 & 7 done)
Week 4	Build LLM Project Intelligence layer (risk classification from text + Q&A interface); integrate SHAP explainability	Working LLM module + explainable predictions
Week 5	Build dashboard front-end; wire all modules together end-to-end; write documentation/model card	End-to-end working prototype
Week 6	Polish UI, record demo video, build pitch deck, rehearse Q&A, stress-test the story against the problem statement text line by line	Final pitch deck + rehearsed demo

If your actual timeline is shorter (many SIH internal hackathons run 36–48 hours for the finale, with weeks of prep before), compress Weeks 1–3 into pre-finale preparation and treat the finale itself as Weeks 4–6 focused on integration, polish, and pitch — judges cannot tell how long you worked, only what you show.

5.4 Suggested Team Role Split (for a typical 6-member SIH team)

- 1 Data/ML lead — owns data pipeline, feature engineering, core ensemble models, evaluation framework
- 1 ML/Stats specialist — owns the Bayesian/HMM early-warning layer and the RCF/statistical baselines (this person becomes your best defense in Q&A on requirement 'b')
- 1 NLP/LLM engineer — owns the Project Intelligence Assistant and text-based risk classification
- 1 Full-stack/dashboard developer — owns the front-end, SHAP visualization, and end-to-end integration
- 1 Domain/documentation lead — owns mapping every feature and finding back to MoSPI's exact problem-statement language, the model card, and deployment documentation
- 1 Presentation/pitch lead — owns the deck, demo script, video, and rehearsing answers to likely judge questions

5.5 Why Teams Actually Lose (Avoid These)

- Building only a notebook with accuracy metrics and no usable interface — judges cannot evaluate what they cannot see working.

- Ignoring the explicit sub-questions in the problem statement (especially the CUF-sufficiency ask) in favor of a generic 'we built a cost predictor' pitch.
- Overclaiming accuracy without honest evaluation against baselines — judges with technical backgrounds will probe this immediately.
- Using paid/closed APIs (e.g., proprietary LLM APIs) when the problem statement explicitly asks for open-source tools — a credibility and compliance risk.
- A pitch that leads with technology ('we used a stacked ensemble with SHAP') instead of leading with the government impact ('this flags a ₹500 crore project's risk of overrun 4 months earlier than current reporting allows').
- No clear articulation of how this plugs into PAIMANA's real monthly data-refresh cycle — leaving the impression it's a one-off analysis, not a monitoring system.

5.6 Demo & Pitch Structure That Consistently Lands Well

1. Open with the cost of the problem in MoSPI's own numbers (₹42.78 lakh crore revised cost across 1,981 projects) — ground the pitch in scale before showing any technology.
2. State your specific, named innovation in one sentence (e.g., 'a predictive early-warning layer that tells administrators not just which projects are at risk, but whether today's CUF data is even enough to know that — and gives them a natural-language assistant to ask why').
3. Live demo: show a live risk dashboard, drill into one flagged project, show the SHAP-based 'why,' then show the LLM assistant answering a natural-language question about it.
4. Show the honest comparison chart: baseline/RCF vs. your ML model's accuracy and lead time — this single slide builds more credibility than any accuracy number alone.
5. Close with deployment realism: how this fits PAIMANA's monthly cycle, what's open-source, and what MoSPI would need to do to pilot it.
6. Prepare 1-line answers in advance for the questions judges are almost certain to ask: 'How is this different from just using EVM/CPI-SPI metrics already in use?', 'What happens if CUF data quality is poor?', and 'Why should we trust an LLM's output for something this consequential?'

5.7 Final Pre-Submission Checklist

- Every technical dimension (a), (b), (c) from the problem statement is explicitly addressed and can be pointed to in your deck/demo
- At least one 'possible expected outcome' from MoSPI's list beyond the core prediction model is visibly built (e.g., risk scoring framework AND early warning alerts AND the LLM assistant)
- Only open-source tools are used, and this is stated explicitly
- A working, clickable/demoable artifact exists — not only slides
- Model performance is reported honestly against at least one non-ML baseline
- Documentation/deployment framework exists as an artifact (even a short PDF/README), matching MoSPI's explicitly listed outcome
- The pitch opens with impact/scale, not technology, and closes with deployment realism

5.8 ML vs. Regression/EVM — Head-to-Head Evidence (New)

Predicting Cost Overrun in Construction Projects Using Machine Learning Algorithms: The Case of Jordan

— *Hamdan & Thneibat (2025) — Engineering, Construction and Architectural Management*

The largest, most rigorous head-to-head comparison in this space (836 public projects, 15 ML algorithms). CatBoost wins (R^2=0.883); variation orders alone account for 41% feature importance. Your single best citation for 'why gradient boosting' and for validating your top feature choices.

DOI: https://doi.org/10.1108/ecam-09-2024-1209

Summary: https://consensus.app/papers/predicting-cost-overrun-in-construction-projects-using-hamdan-thneibat/ea3f163bcb0c5e2482e7d32a7100543b/

Optimizing Project Time and Cost Prediction Using a Hybrid XGBoost and Simulated Annealing Algorithm

— *ForouzeshNejad, Arabikhan & Aheleroff (2024) — Machines*

Your strongest quantified evidence of ML beating EVM directly: ~50% lower cost error and ~80% lower time error than EVM/ESM. Cite this exact statistic in your Phase 7 evaluation slide.

DOI: https://doi.org/10.3390/machines12120867

Summary: https://consensus.app/papers/optimizing-project-time-and-cost-prediction-using-a-hybrid-forouzeshnejad-arabikhan/92406ec8f2e359c48cc90db62b930de8/

Comparison of Artificial Intelligence Techniques for Project Conceptual Cost Prediction: A Case Study and Comparative Analysis

— *Elmousalami (2019) — IEEE Trans. on Engineering Management (94 citations)*

Rigorously benchmarks 20 AI/statistical techniques head-to-head at the conceptual (earliest) project stage — the closest analogue to a newly sanctioned PAIMANA project with minimal history. XGBoost wins, but the paper's real value is its systematic method for comparing so many techniques fairly.

DOI: https://doi.org/10.1109/tem.2020.2972078

Summary: https://consensus.app/papers/comparison-of-artificial-intelligence-techniques-for-elmousalami/2dc644dd90ee5215857118c08b78de5f/

Predicting Project Cost Overrun Levels in Bidding Stage Using Ensemble Learning

— *Moon & Williams (2020) — Journal of Asian Architecture and Building Engineering*

The essential counter-evidence paper: at the earliest, sparsest-data project stage, even an ensemble model only reached 61.4% accuracy. Cite this to show you understand ML's real limits and to justify blending ML with Reference Class Forecasting for newly sanctioned projects.

DOI: https://doi.org/10.1080/13467581.2020.1765171

Summary: https://consensus.app/papers/predicting-project-cost-overrun-levels-in-bidding-stage-moon-williams/593d5b06c86e50638d9b435438a834da/

Cost Overrun Risk Assessment and Prediction in Construction Projects: A Bayesian Network Classifier Approach

— *Ashtari & Ansari (2022) — Buildings (54 citations)*

Strong evidence base for your Phase 4 early-warning layer: Bayesian Networks reached 78.86% accuracy by modeling interdependencies between risk factors, significantly beating Naive Bayes and Decision Tree baselines that ignore those interactions.

DOI: https://doi.org/10.3390/buildings12101660

Summary: https://consensus.app/papers/cost-overrun-risk-assessment-and-prediction-in-ashtari-ansari/2dde8125db1151c0950258d1563d69f0/

6. Priority Reading List — Must-Read Papers

Selected from the full 100-paper corpus based on relevance, methodological importance, and direct applicability to the phases above. Organized by the part of the solution each paper informs. DOI links are given (resolve via https://doi.org/<DOI>); Consensus.app summary links are also provided where available.

5.1 Foundational Reviews — Read These First

Advancement of AI in Cost Estimation for Project Management Success: A Systematic Review of ML, DL, Regression, and Hybrid Models

— *Shamim, Hamid, Nyamasvisva & Rafi (2025) — Modelling*

Best single starting point. Synthesizes 39 studies and gives concrete accuracy benchmarks by model family (DL 85–90%, ML 75–80%, regression 70–80%, hybrid 80–90%) — use these numbers to set realistic performance targets.

DOI: https://doi.org/10.3390/modelling6020035

Summary: https://consensus.app/papers/advancement-of-artificial-intelligence-in-cost-shamim-hamid/568f3e97fc865498b5a5bf70fbc80ba5/

Predictive Analytics in Construction Scheduling Using Machine Learning Algorithms

— *Ingole (2026) — Int'l Journal of Creative and Open Research in Eng. & Mgmt.*

PRISMA-based review of 30 studies specifically on schedule/delay prediction; confirms Random Forest and XGBoost as consistent top performers (85%+ accuracy) and proposes a BIM+IoT+ML integration framework relevant to your dashboard design.

DOI: https://doi.org/10.55041/ijcope.v2i7.142

Summary: https://consensus.app/papers/predictive-analytics-in-construction-scheduling-using-ingole/659e612c1e82571e890379a1f1bb6bfb/

5.2 Cost Overrun Prediction — Model Architecture References

A Machine Learning Study to Improve the Reliability of Project Cost Estimates

— *Narbaev, Hazır, Khamitova & Talgat (2023) — Int'l Journal of Production Research*

Real-data XGBoost model (110 projects, 1,268 data points) benchmarked against EVM and multiple ML baselines; directly usable as your Phase 3 model template, including its early-warning framing.

DOI: https://doi.org/10.1080/00207543.2023.2262051

Summary: https://consensus.app/papers/a-machine-learning-study-to-improve-the-reliability-of-narbaev-haz%C4%B1r/e8e797e8f30f5fc29565b9701a1c81c6/

A Hybrid Ensemble Learning Approach for Intelligent Cost Prediction in Public Construction Projects

— *Lyu, Tu & Yu (2026) — Proc. Int'l Conf. on AI Decision-Making and Management*

Best-in-corpus stacking architecture: XGBoost + LightGBM + CatBoost + Random Forest with Ridge meta-learner and SHAP, achieving R^2=0.971 on public-sector data. This is the closest template to your public-infrastructure use case — read this before building Phase 3.2.

DOI: https://doi.org/10.1145/3811839.3811868

Summary: https://consensus.app/papers/a-hybrid-ensemble-learning-approach-for-intelligent-cost-lyu-tu/8ae8121649f35b8da2734269c9c3d097/

Determining Cost and Causes of Overruns in Infrastructure Projects in South Asia

— *Andrić, Lin, Cheng & Sun (2024) — Sustainability*

Most geographically relevant paper in the corpus (South Asian region, 138 infrastructure projects). Reports average

overrun of 3.3% and validates Random Forest regression as best-fit — a strong regional benchmark for calibrating your PAIMANA model's expectations.
DOI: https://doi.org/10.3390/su162411159
Summary: https://consensus.app/papers/determining-cost-and-causes-of-overruns-in-infrastructure-andri%C4%87-lin/f20096fef43159808db260e911eeb762/

Developing an Integrative Data Intelligence Model for Construction Cost Estimation
— *Ali, Burhan et al. (2022) — highly cited (39 citations)*
XGBoost-RF hybrid identifying inflation as the dominant cost driver — useful evidence for prioritizing macroeconomic enrichment variables in your Phase 1 feature set.
DOI: https://doi.org/10.1155/2022/4285328
Summary: https://consensus.app/papers/developing-an-integrative-data-intelligence-model-for-ali-burhan/241fb5de28915b7785cc9dc5b09577a3/

5.3 Time-Varying Early Warning — Bayesian / Markov Models

Project Cost Overrun Risk Prediction Using Hidden Markov Chain Analysis
— *Leu & Liu (2023) — Buildings*
Real-time HMM model producing early alerts and suggested cost-management actions — the closest existing template for your Phase 4 'evolving risk state' design.
DOI: https://doi.org/10.3390/buildings13030667
Summary: https://consensus.app/papers/project-cost-overrun-risk-prediction-using-hidden-markov-leu-liu/b1566d639c995a94b19ad3c9d91ad1bc/

Dynamic-Bayesian-Network-Based Project Cost Overrun Prediction Model
— *Leu & Lu (2023) — Sustainability*
Bayesian Network approach to continuously updating cost-overrun risk as new project data arrives — directly informs the state-transition design (Low → Watch → High-Risk → Critical) recommended in Phase 4.
DOI: https://doi.org/10.3390/su15054570
Summary: https://consensus.app/papers/dynamicbayesiannetworkbased-project-cost-overrun-leu-lu/61f2561e5537505fbf41b7f74dd2b969/

Probabilistic Risk Assessment Framework for Cost Overruns Predictions in Infrastructure Projects Using Randomized Simulations (PRIMoS)
— *Canesi & Gabrielli (2025) — Computer-Aided Civil and Infrastructure Engineering*
A more advanced simulation-based uncertainty-quantification framework — useful if you want to go beyond point predictions to full probability distributions for each project's likely final cost.
DOI: https://doi.org/10.1111/mice.70100
Summary: https://consensus.app/papers/probabilistic-risk-assessment-framework-for-cost-canesi-gabrielli/176df085ef1753a7b1bd84acad883dfe/

5.4 LLMs & NLP — For the Project Intelligence Assistant

Large Language Models for Construction Risk Classification: A Comparative Study
— *Erfani & Khanjar (2025) — Buildings*
The single most important paper for Phase 5. Benchmarks 12 model configurations (TF-IDF, Word2Vec, BERT, GPT-4 zero/few-shot) for classifying risk from unstructured text; GPT-4 few-shot (F1=0.81) is competitive with trained BERT+SVM (F1=0.86) with zero labeled data — proving your LLM layer is feasible on a hackathon timeline.

DOI: https://doi.org/10.3390/buildings15183379
Summary: https://consensus.app/papers/large-language-models-for-construction-risk-erfani-khanjar/00eefeaafead5475839d985d2bfa27f5/

Construction Contract Risk Identification Based on Knowledge-Augmented Language Model
— *Wong & Zheng (2023) — arXiv (65 citations)*
Combines LLMs with domain knowledge to identify contractual risk, emulating expert review — a strong reference architecture if you extend the LLM layer to contract/agreement text, not just remarks fields.
DOI: https://doi.org/10.48550/arxiv.2309.12626
Summary: https://consensus.app/papers/construction-contract-risk-identification-based-on-wong-zheng/fb1a378a490a54338c9062f8a5c89086/

5.5 Reference Class Forecasting — Your 'Conventional Method' Benchmark

Curbing Cost Overruns in Infrastructure Investment
— *Park (2021) — European Journal of Transport and Infrastructure Research*
Empirically shows RCF cut UK infrastructure cost overruns from 38% to 5% after adoption vs. the U.S. without it. Essential reading — this is the credible non-ML baseline you should implement and report against in Phase 2 and 7.
DOI: https://doi.org/10.18757/ejtir.2021.21.2.5504
Summary: https://consensus.app/papers/curbing-cost-overruns-in-infrastructure-investment-park/d39e29ef0bb85bdeb722187a2beb973e/

Reference Class Forecasting: Promises, Problems, and a Research Agenda Moving Forward
— *Cantarelli & Davis (2025) — Transportation Planning and Technology*
A critical, up-to-date look at RCF's limitations — read this alongside the Park (2021) paper so your benchmark section presents a balanced, evaluator-credible view rather than one-sided advocacy.
DOI: https://doi.org/10.1080/09537287.2025.2578708
Summary: https://consensus.app/papers/reference-class-forecasting-promises-problems-and-a-cantarelli-davis/bad34d0cfbea57009bd19d232dd05ca4/

5.6 Cost & Delay Drivers — For Feature Engineering & the Driver-Analysis Module

The Cost Drivers of Infrastructure Projects: Definition, Classification, and Conceptualization
— *Watton & Unterhitzenberger (2023) — Journal of Construction Engineering and Management*
Provides a full taxonomy/conceptual framework of infrastructure cost drivers — use this to structure and validate your feature list and your 'Cost Escalation Driver Analysis Module.'
DOI: https://doi.org/10.1061/jcemd4.coeng-13411
Summary: https://consensus.app/papers/the-cost-drivers-of-infrastructure-projects-definition-watton-unterhitzenberger/22e5c97d8173538cbcdb742230eb8f02/

Early Detection of Construction Project Risks in Saudi Arabia: A Mixed-Methods Study on Warning Signs and Mitigation
— *Alsulami & Al-Shaery (2026) — Scientific Reports*
Identifies concrete, practitioner-validated early warning signs (weak project definition, ineffective leadership, inaccurate costing) — directly useful for choosing which qualitative LLM-detected flags to surface in your alert system.
DOI: https://doi.org/10.1038/s41598-026-45775-9

Summary: https://consensus.app/papers/early-detection-of-construction-project-risks-in-saudi-alsulami-al-shaery/06ad620d0fc25c8a96db71e39bbf8e1a/

5.7 Explainability & Uncertainty — For Your Dashboard Layer

Uncertainty-Aware and Explainable Construction Cost Prediction Using a Hybrid Probabilistic Learning Model (NGBoost-ETR)

— *Chen, Khalid et al. (2026) — Scientific Reports*

State-of-the-art example of combining high accuracy (R^2=0.987) with calibrated uncertainty intervals and SHAP interpretability in one framework — the benchmark to aim for in Phases 3 and 6.

DOI: https://doi.org/10.1038/s41598-026-44904-8

Summary: https://consensus.app/papers/uncertainty-aware-and-explainable-construction-cost-chen-khalid/56096cea6ae1506ca4c1805a8e7606a7/

Search-Guided Regression Ensembles for Accurate, Interpretable, and Uncertainty-Aware Construction Cost Estimation

— *Chen & Lim (2026) — Scientific Reports*

Another 2026 explainability-first ensemble framework; useful second reference to cross-check design choices for the SHAP-based dashboard.

DOI: https://doi.org/10.1038/s41598-026-51706-5

Summary: https://consensus.app/papers/searchguided-regression-ensembles-for-accurate-chen-lim/a3f0316dc6655afab82c928989863181/

5.8 Integrated / End-to-End Frameworks — Closest Analogues to Your Full System

An Integrated Machine Learning Framework for Predictive and Sustainable Road Construction Project Management

— *Hashemi & Salehy (2026) — SJMARS*

Predicts cost, duration, risk level, AND completion percentage in one framework — structurally the closest existing analogue to the full multi-output PAIMANA system you are being asked to build.

DOI: https://doi.org/10.55544/sjmars.5.3.5

Summary: https://consensus.app/papers/an-integrated-machine-learning-framework-for-predictive-hashemi-salehy/a2efaac2692b53e6a100236329eb8040/

Integrating Machine Learning and Network Analytics to Model Project Cost, Time and Quality Performance

— *Uddin & Ong (2023) — Production Planning & Control*

Combines ML with network analytics to jointly model cost, time, and quality — useful if you want your risk score to incorporate quality/defect signals alongside cost and schedule.

DOI: https://doi.org/10.1080/09537287.2023.2196256

Summary: https://consensus.app/papers/integrating-machine-learning-and-network-analytics-to-uddin-ong/fcb853ccc8ac5844b20ae9c7624c6517/

Integrated Assessment of Environmental, Infrastructural and Social Risks for Urban Public Safety

— *Liu et al. (2026) — Scientific Reports*

GIS + MCDA + ML integrated risk-assessment framework reducing composite risk by 22–30% — a good reference if you extend beyond cost/schedule into broader implementation/social risk scoring.

DOI: https://doi.org/10.1038/s41598-026-35822-w

Summary: https://consensus.app/papers/integrated-assessment-of-environmental-infrastructural-liu/c66ccf36cc7d5b1e9e2b945290c6a627/

7. Full Paper Corpus — Category Index

For reference, the complete 100-paper set (as provided) breaks down into the following categories. Use this to locate additional supporting papers beyond the priority list above if your team splits up the workload by module.

Category	Approx. Count	Primary Use in This Project
Cost overrun prediction (ML/DL/hybrid)	~30	Phase 3 model design; Phase 7 benchmarking
Schedule/time overrun & delay prediction	~20	Phase 3 schedule models
Risk scoring, classification & early warning	~15	Phase 4 risk scoring & alerting
Causal/factor analysis of overruns	~15	Phase 1 feature engineering; driver-analysis module
LLMs & NLP for construction	~5	Phase 5 Project Intelligence Assistant
Data infrastructure / dashboards / open data	~6	Phase 6 dashboard design
Reference Class Forecasting	~5	Phase 2 & 7 conventional-method benchmark

Note: Category counts are approximate, based on thematic grouping of paper titles/abstracts, and some papers span more than one category.